

How a Unified Development Toolchain Supports Automated Testing Consistency in a TypeScript Full-Stack Monorepo: A Case Study of Circles

Aldiyar Seylkhonov

*Bachelor Degree Student, School of
Software Engineering
Astana IT University (AITU)
Astana, Kazakhstan
seylkhonov.aldiyar@gmail.com*

Alexandr Tyulkov

*Master Degree Student, School of
Software Engineering
Astana IT University (AITU)
Astana, Kazakhstan
widesehl@gmail.com
0009-0009-6422-0559*

Malika Ishakhanova

*Master Degree Student, School of
Software Engineering
Astana IT University (AITU)
Astana, Kazakhstan
malikaishakhanova@gmail.com*

Abstract—Quality assurance in full-stack TypeScript monorepos is difficult because frontend, backend, shared packages, and continuous integration pipelines can fail at different technical boundaries. This paper studies the Circles repository as a case study and addresses a specific gap: risk-based test planning, automated execution, quality gates, and experimental evidence are often documented separately rather than as one reproducible QA argument. The study proposes a risk-based automated testing strategy for Circles. The strategy maps system risks to unit, integration, end-to-end, performance, mutation, and chaos tests, then evaluates the resulting pipeline through repository inspection and direct execution. The observed automation baseline includes 33 passing backend tests, 7 backend test files, 2 GitHub Actions workflows, and a selected-helper mutation score of 87.93%. Local performance testing reached 1,523.90 requests per second for the authenticated library overview endpoint with 12.62 ms average latency. Chaos testing showed that a PostgreSQL outage produced a user-visible HTTP 500 response, but the service recovered after the database restarted. These findings show that risk-based automation improves traceability and exposes concrete gaps, but current evidence remains limited by low whole-backend coverage and a local synthetic dataset.

Index Terms—monorepo, TypeScript, unified toolchain, automated testing, Vite, software engineering

I. INTRODUCTION

Full-stack web systems increasingly combine user interfaces, API services, background workflows, databases, and third-party integrations in one delivery pipeline. Quality assurance is important in this setting because a failure in one layer can affect authentication, data correctness, user trust, and deployment confidence. Automated tests help reduce this risk, but automation is only useful when it is linked to clear risk priorities and measured evidence. Without that link, teams can collect passing tests while important system risks remain untested.

This paper studies Circles, a TypeScript full-stack monorepo for Spotify listening-history analytics. The system includes a React frontend, a Hono backend, PostgreSQL storage through Drizzle ORM, background import workflows, and shared utility packages. Users authenticate through Spo-

tify OAuth, import listening data, and view statistics such as top artists, listening time, range-filtered summaries, leaderboards, and library overviews. The repository uses Vite+ as a unified command interface for development, testing, formatting, linting, and builds. The main QA-relevant complexity comes from authentication, external data ingestion, database-backed queries, workspace-level test configuration, and continuous integration.

Testing Circles is difficult for four reasons. First, Spotify OAuth and session handling are integration-heavy and can block every authenticated user flow. Second, import and parsing errors can silently corrupt listening statistics before a user notices the defect. Third, the frontend, backend, and shared packages use different runtime assumptions even though they share one monorepo. Fourth, performance and availability risks cannot be detected by ordinary unit tests alone. These risks require a strategy that connects critical components to the right test type and metric.

Despite the adoption of Vite+ and automated tests, there is a lack of integrated evidence connecting risk-based prioritization, automated execution, quality gates, and experimental validation. This creates a negative consequence: narrow test suites can pass while controller behavior, workflow orchestration, database failure handling, and full-pipeline consistency remain weakly supported.

This work aims to design and evaluate a risk-based automated QA framework for the Circles monorepo. The study connects planning evidence from earlier assignments with direct experiments on coverage, execution time, mutation score, performance, and failure recovery.

The paper makes four contributions:

- Proposed a risk-based testing strategy tailored to a TypeScript full-stack monorepo.
- Mapped high-priority risks to automated test types, quality gates, and measurable metrics.
- Evaluated the current Circles automation pipeline using test execution, coverage, performance, mutation, and chaos evidence.

- Identified remaining QA gaps in backend integration coverage, frontend test orchestration, and database dependency handling.

II. LITERATURE REVIEW

The literature review is organized around four topics: risk-based testing, automated testing in continuous integration, quality metrics, and advanced testing methods. The goal is not to list papers one by one. The goal is to identify how existing work supports the design of the Circles QA strategy and where the project still fills a practical gap.

A. Risk-Based Testing and Software Risk

Risk-based testing gives priority to functions and scenarios that have higher failure probability or higher business impact. This principle is consistent with software risk management, where risk exposure is commonly modeled through likelihood and consequence [1]; [2]. In testing, this means that test effort should not be distributed evenly across all files. It should be concentrated on authentication, data ingestion, API contracts, and other components where failure has higher user impact.

Earlier risk-based testing studies show that prioritization improves the efficiency of defect detection when risk analysis is explicit and traceable [3]; [4]. However, risk models are often documented as planning artifacts and are not always connected to pipeline enforcement. Circles addresses this gap by mapping each high-risk area to a test type, automation level, quality gate, and observed metric.

B. Test Automation and CI/CD

Automated testing is most useful when it becomes part of continuous integration rather than a separate manual activity. Studies of continuous integration show that frequent automated checks can improve feedback speed, but they also introduce cost, flakiness, and configuration complexity [5]; [6]; [7]. Wang et al. found that higher test automation maturity is associated with better product quality and shorter release cycles [8]. Yu et al. also show that non-functional testing in CI depends on tool support, metrics, and team practice [9].

This literature supports the use of Vite+ as a common command interface in Circles. A unified command surface reduces the number of ways tests can be executed, which improves reproducibility. However, CI research also warns that passing commands are not enough. A pipeline must still test representative paths and must expose failures at integration boundaries.

Automated test generation and amplification research also supports a staged approach. REST API test generation can improve endpoint coverage, but generated tests still need useful oracles and realistic input structures [10]. Developer-centered test amplification can strengthen existing tests, but human review remains important when generated or expanded tests expose ambiguous behavior [11]. This is relevant to Circles because the most useful next tests are not random additional assertions. They are targeted route, middleware, and database checks tied to known risks.

C. Quality Metrics and Test Effectiveness

Coverage is a common QA metric, but the literature warns against treating coverage as proof of quality. Inozemtseva and Holmes found that coverage alone has a weak relationship with test suite effectiveness when suite size is controlled [12]. This is important for Circles because selected helper modules can reach 100% coverage while the backend as a whole remains below the global coverage threshold.

Mutation testing is a stronger adequacy signal because it checks whether tests detect small code changes. Jia and Harman describe mutation testing as a mature fault-based testing method, while later surveys show that mutation testing remains useful but expensive [13]; [14]. This supports the use of Stryker on selected backend helper modules. It also limits the interpretation: a high mutation score for four helper files cannot prove full-system quality.

D. Advanced Testing Approaches

Performance testing and resilience testing extend QA beyond functional correctness. Load-testing research shows that performance behavior depends on workload, environment, and data shape [15]; [16]. Chaos engineering studies show that deliberate failure injection can reveal resilience weaknesses that normal tests miss [17]; [18]. These approaches are relevant to Circles because authenticated data routes depend on PostgreSQL and session lookup.

TypeScript-specific research adds another reason to test configuration boundaries. Bogner and Merkel found that TypeScript projects improve some quality indicators but do not automatically reduce all bug outcomes [19]. Tang et al. show that TypeScript failures often occur around toolchains and configuration rather than only application logic [20]. This means a TypeScript monorepo still needs explicit QA evidence for build, test, and runtime composition.

The reviewed literature shows a clear gap. Existing work supports risk-based prioritization, CI automation, coverage, mutation testing, performance testing, and chaos engineering. Less work shows how these activities can be connected in one coursework-scale QA case study for a full-stack TypeScript monorepo. This project addresses that gap by integrating risk, automation, quality gates, and experiments into one research narrative.

III. METHODOLOGY

The methodology connects the earlier QA assignments into one pipeline. Assignment 1 provides the risk strategy. Assignment 2 provides the automation framework and quality gates. Assignment 3 provides performance, mutation, and chaos experiments. The midterm report provides evidence-based risk revision. The resulting pipeline is:

- Risk identification and scoring.
- Test design based on risk priority.
- Automated execution through Vite+ and CI.
- Quality gates for pass, coverage, execution time, and static checks.
- Experimental measurement through coverage, load, mutation, and failure injection.

A. Mapping from Previous Work

The paper does not copy the earlier assignments directly. Each previous submission is transformed into a research-paper role. The first assignment is used as the formal risk model. The second assignment is used as the implementation baseline for automated tests and quality gates. The third assignment is used as the experimental evidence source. The midterm report is used as the empirical correction step because it revises the original assumptions using observed behavior.

TABLE I
EXPLICIT MAPPING FROM PREVIOUS ASSIGNMENTS TO THE PAPER

Previous work	Paper location	Transformation
Assignment 1	Methodology	Risk descriptions become a scored and traceable risk strategy
Assignment 2	Methodology and results	Test scripts and gates become an evaluated automation framework
Assignment 3	Results and discussion	Raw experiments become performance, mutation, and chaos evidence
Midterm	Results and discussion	Revised risks and edge cases become evidence-backed limitations

B. Risk-Based Testing Strategy

Risks were identified through architecture analysis, user-flow impact, implementation complexity, and earlier test observations. The scoring model is:

$$\text{Risk Score} = \text{Likelihood} \times \text{Impact} \quad (1)$$

Likelihood and impact are scored from 1 to 5. Likelihood estimates the chance of failure based on complexity, dependency count, and observed behavior. Impact estimates the user-facing or system-level consequence if the component fails.

TABLE II
RISK LIST USED FOR TEST PRIORITIZATION

Risk	Description	Component	L	I	Score
R1	Sign-in or session failure blocks authenticated access	OAuth and sessions	4	5	20
R2	Incorrect import parsing corrupts listening statistics	Spotify export ingestion	4	5	20
R3	Wrong range filtering produces misleading dashboard data	Dashboard and library queries	3	5	15
R4	API contract or middleware error breaks route behavior	Backend controllers	3	4	12
R5	Pipeline composition failure hides package-level inconsistency	Frontend test orchestration	3	4	12
R6	Database outage causes authenticated requests to fail	PostgreSQL dependency	3	5	15

The midterm evidence revised two risks upward. Data ingestion increased because edge-case tests found permissive parsing and incomplete field validation. API correctness increased because controller-level and middleware paths remained weakly covered.

The risk model is intentionally simple. A more complex model could include cost, detectability, and historical defect frequency. Those values were not available consistently across the repository. The likelihood times impact model was therefore selected because it is reproducible from the available evidence and easy to defend during review.

C. Risk to Test Mapping

Each risk is mapped to at least one test type and one metric. This prevents a common weakness in QA plans: risk tables that do not affect test execution.

TABLE III
MAPPING FROM RISKS TO TESTS AND AUTOMATION LEVEL

Risk	Test type	Test description	Automation
R1	E2E and chaos	Session creation, authenticated route access, database outage behavior	Partial
R2	Unit and mutation	Spotify export validation, URI parsing, boundary cases	Full
R3	Unit and performance	Range helper checks and authenticated library overview load test	Full
R4	Integration	HTTP contract and middleware behavior	Planned
R5	CI execution	Aggregated versus isolated frontend test runs	Partial
R6	Chaos	PostgreSQL stop and restart during authenticated request path	Full

The mapping also separates current evidence from planned evidence. R2 and R3 have strong helper-level automation because their behavior is expressed in pure functions. R1, R4, R5, and R6 require integration or environment-level checks because they involve authentication, routing, CI composition, or database availability. This distinction is important because not every high-risk component can be tested well with the same method.

The selected strategy therefore follows a layered test design. Unit tests are used for deterministic parsing, date conversion, retry, and ZIP signature logic. Integration tests are reserved for route and middleware contracts. End-to-end tests are reserved for user-visible browser flows. Mutation testing checks selected unit-test strength. Performance testing checks latency and throughput under local load. Chaos testing checks behavior when a required dependency fails.

D. Automation Framework

Circles uses Vite+ as the main automation entry point. Backend, frontend, and shared packages use one command interface, while package-specific configurations handle browser tests and end-to-end tests. The backend unit suite uses Vitest through Vite+. Playwright covers browser-level flows. Stryker is configured for mutation testing on selected backend helper modules. Autocannon is used for local performance testing.

TABLE IV
AUTOMATION TOOLS AND ROLES

Tool	Role	Reason for selection
Vite+	Unified command interface	Matches repository workflow and reduces per-package command differences
Vitest	Unit and component tests	Already integrated through Vite+ and suitable for TypeScript modules
Playwright	End-to-end browser checks	Supports real browser execution for user flows
Stryker	Mutation testing	Measures whether selected tests detect code changes
Autocannon	Performance testing	Provides repeatable local HTTP load tests
GitHub Actions	CI enforcement	Runs checks on push and pull request events

The CI model follows a staged flow: install dependencies, run static checks, execute recursive tests, then run build or end-to-end checks. This structure favors fast feedback before slower browser and experimental checks. The trade-off is that full performance, mutation, and chaos experiments are not run on every commit because they require more time and local infrastructure.

TABLE V
AUTOMATION PIPELINE AND EVIDENCE FLOW

Pipeline stage	Executed checks	Primary evidence produced
Commit or PR	vp check, recursive tests	Static correctness and regression pass/fail
Package validation	Backend, frontend, and shared package tests	Execution consistency across workspaces
Coverage run	Backend coverage command	Covered scope and failed global threshold
Experimental run	Autocannon, Stryker, PostgreSQL outage	Performance, mutation, and resilience metrics
Review	Manual analysis of reports	Limitations, trade-offs, and future work

E. Quality Gates

Quality gates define whether the pipeline can proceed. The gates are intentionally simple because the project is still in a coursework-scale stage.

TABLE VI
QUALITY GATES AND FAILURE ACTIONS

Metric	Threshold	Action if failed
Critical tests	100% pass	Block merge until the failing test is fixed
Critical-module coverage	$\geq 80\%$	Add tests or reduce unsupported claims
Backend execution time	≤ 5 min	Inspect slow tests or async waits
Static checks	0 major violations	Block merge through vp check
Mutation score on selected helpers	$\geq 80\%$	Add targeted tests for surviving mutants
Read endpoint latency	P99 ≤ 200 ms	Investigate query and database behavior

The thresholds are not arbitrary. The pass-rate gate protects the existing regression baseline. The coverage gate comes from the earlier automation plan and is used as a signal that high-risk modules need broader tests. The 5 minute execution-time gate is much higher than the current backend suite time, which leaves room for future integration tests. The mutation score gate follows the common high-threshold interpretation used by mutation-testing tools. The 200 ms read-latency gate is selected as a user-facing responsiveness target for dashboard and library views.

F. Experimental Setup

Experiments were executed locally against the Circles repository using supported commands. The runtime environment used Vite+ vp v0.1.18, Node.js v24.15.0, PostgreSQL 16, and a dedicated local database container on port 5433. The seeded dataset contained 7 users, 30 tracks, and 16,600 history rows. The authenticated tests used the built-in E2E login endpoint so that Spotify OAuth did not block reproducibility.

Performance testing used a live backend server on port 8001, 20 concurrent connections, and 10-second runs. Mutation testing used the backend Stryker configuration and targeted four helper files: range.ts, retry.ts, spotify-export.ts, and zip.ts. Chaos testing stopped and restarted PostgreSQL while the backend process remained running.

The experiment can be reproduced by installing dependencies with vp install, running checks with vp check, executing backend tests with vp run @circles/backend#test, running coverage and mutation commands from the backend package scripts, starting the local backend and PostgreSQL container, then repeating the documented Autocannon and database outage steps.

The data used in the experiments is synthetic. This choice avoids privacy risk from real Spotify history and keeps the experiment repeatable. The cost is lower realism. The dataset is large enough to exercise query paths, but it does not represent long-term production traffic, many concurrent users, or rare Spotify export formats.

IV. RESULTS

This section reports observed data only. Interpretation is reserved for the discussion.

A. Test Coverage Versus Risk Priority

TABLE VII
COVERAGE ALIGNMENT BY RISK PRIORITY

Risk	Priority	Coverage	Test types applied
R1	High	Partial	E2E login support, authenticated route checks, chaos dependency test
R2	High	100% selected scope	Unit, edge-case, invalid-input, mutation
R3	High	100% selected scope	Unit range tests, performance test
R4	High	0% direct scope	Controller tests planned, routing behavior observed during performance setup
R5	High	Partial	Frontend aggregated and isolated execution comparison
R6	High	Scenario covered	PostgreSQL stop and restart chaos test

The backend coverage command executed 33 tests successfully. The global backend coverage remained below configured thresholds: 5.88% statements, 6.83% branches, 5.45% functions, and 5.17% lines. The selected helper modules reached full local coverage in the earlier targeted reports.

TABLE VIII
COVERAGE BY MEASUREMENT SCOPE

Scope	Stmt.	Branch	Func.	Lines	Gate status
Selected helper scope	100%	100%	100%	100%	Pass in targeted reports
Whole backend	5.88%	6.83%	5.45%	5.17%	Fail global threshold

B. Defect Detection: Manual Versus Automated

TABLE IX
OBSERVED FINDINGS BY DETECTION METHOD

Method	Total findings	Critical findings	Detection stage
Manual architecture review	2	0	Assignment 1 risk planning
Automated unit tests	3 design behaviors	0	Midterm edge-case execution
Coverage analysis	1 gate failure	0	Assignment 3 baseline validation
Performance setup	1 route behavior	0	Assignment 3 load-test preparation
Chaos experiment	1 outage behavior	1	Assignment 3 failure injection

TABLE X
AUTOMATED AND EXPERIMENTAL FINDINGS

Test type	Findings	Severity	Example
Unit and edge tests	3	Medium	Permissive URI parsing and weak ZIP signature acceptance
Coverage command	1	High	Global backend coverage below configured gate
Performance test setup	1	Medium	/leaderboard required authentication during execution
Chaos test	1	High	Authenticated route returned HTTP 500 during database outage

C. Execution Time Analysis

TABLE XI
MANUAL VERSUS AUTOMATED EXECUTION TIME

Test stage	Manual estimate	Automated time
Backend baseline suite	10-15 min	95 ms
Backend extended suite	20-30 min	206 ms
Coverage command	Not practical manually	Tests pass, threshold fails
Mutation testing	Not practical manually	29 s
Performance test	Not practical manually	10 s per endpoint

Direct package execution also produced the following observed outcomes: backend unit tests passed with 7 files and 33 tests during Assignment 3, while the earlier paper inspection recorded stable backend and shared-package execution and a frontend aggregated-run failure.

TABLE XII
OBSERVED TEST EXECUTION OUTCOMES

Run scope	Files	Tests	Result	Notes
Backend baseline	4	21	Pass	Assignment 2 helper suite
Backend extended	7	33	Pass	Assignment 3 and midterm suite
Shared utils	1	1	Pass	Package-level utility check
Frontend unit isolated	1	4	Pass	Dedicated unit project
Frontend aggregated	1 pass, 1 fail	2 pass	Fail	Runner initialization failure in combined project run

D. Performance Testing Results

TABLE XIII
LOCAL LOAD-TEST RESULTS WITH 20 CONCURRENT CONNECTIONS OVER 10 SECONDS

Scenario	Auth	Avg latency	P99	Avg RPS
/health normal baseline	No	0.01 ms	0 ms	43,093.82
/leaderboard? period=all aggregate read	Yes	13.83 ms	24 ms	1,396.70
/library/overview? range=30d authenticated read	Yes	12.62 ms	21 ms	1,523.90

All measured read scenarios stayed below the informal 200 ms P99 target in the local experiment.

TABLE XIV
PERFORMANCE QUALITY-GATE SUMMARY

Metric	Target	Best observed	Worst observed	Status
P99 latency	<= 200 ms	0 ms	24 ms	Pass
Average latency	Report only	0.01 ms	13.83 ms	Recorded
Throughput	Report only	1,396.70 RPS	43,093.82 RPS	Recorded
Error behavior	0 critical route defects	n/a	Auth required on selected public route	Finding

E. Mutation Testing Results

TABLE XV
MUTATION TESTING RESULTS FOR SELECTED BACKEND HELPERS

File	Total	Killed	Survived	Time-out	Score
range.ts	8	8	0	0	100.00%
retry.ts	26	19	6	1	76.92%
spotify-export.ts	16	16	0	0	100.00%
zip.ts	8	7	1	0	87.50%
All selected files	58	50	7	1	87.93%

The mutation run generated 58 mutants and completed in 29 seconds. The selected helper scope passed the 80% mutation-score gate overall, but retry.ts alone stayed below that threshold.

F. Chaos Testing Outcomes

TABLE XVI
CHAOS EXPERIMENT AGAINST POSTGRESQL AVAILABILITY

Experiment	Failure injected	System behavior	Recovery
Baseline request	None	/library/overview? range=30d returned HTTP 200	n/a
Database outage	PostgreSQL stopped	Endpoint returned HTTP 500 with session lookup failure	Not available during outage
Database restart	PostgreSQL started	Same endpoint returned HTTP 200 again	Recovered without API restart

TABLE XVII
QUALITY GATE OUTCOMES

Gate	Threshold	Observed	Status
Critical tests	100% pass	33/33 backend tests passed	Pass
Critical-module coverage	$\geq 80\%$	57% high-risk module coverage in midterm	Partial
Whole-backend coverage	Configured high threshold	Below 7%	Fail
Mutation score	$\geq 80\%$	87.93% selected helper score	Pass
Read latency	P99 ≤ 200 ms	Worst observed P99 24 ms	Pass
Database availability	No user-visible critical error	HTTP 500 during outage	Fail

V. DISCUSSION

The results show that the risk-based strategy worked best where risks were mapped to small, deterministic units. The Spotify export parser, range helper, retry helper, and ZIP guard all became measurable through automated tests. These modules produced fast execution, selected-scope coverage, and mutation evidence. This pattern supports the value of starting with high-risk helper functions when a system does not yet have broad integration coverage.

The results also show that high local coverage can be misleading. Four helper modules reached full selected-scope coverage, but global backend coverage stayed below 7%. The cause is scope mismatch. The test suite covers helper logic more strongly than controllers, middleware, workflow orchestration, and database-backed behavior. Therefore, the coverage gate did not fail because individual helper tests were weak. It failed because the automated scope was too narrow.

Automation improved feedback speed. Backend test execution took milliseconds, while manual checking of the same boundaries would require repeated setup and manual input

construction. Mutation testing gave a deeper signal than coverage for the selected helpers. The 87.93% score indicates that most injected changes were detected, but the surviving mutants in `retry.ts` and `zip.ts` show that some boundary behavior remains under-specified.

The performance results were strong under local conditions. The authenticated library overview endpoint stayed near 13 ms average latency and below 25 ms P99. This happened because the experiment used a small synthetic dataset and a local database. The result is useful as a baseline, but it does not prove production behavior under remote network latency, larger datasets, or concurrent real users.

The chaos experiment exposed the clearest user-visible failure. When PostgreSQL stopped, the authenticated endpoint failed during session lookup and returned HTTP 500. The service recovered after the database restarted without restarting the API. This is a mixed result. Process-level recovery worked, but user-visible availability did not. The failure point also means that every authenticated route depends on database availability, not only the tested library route.

A. Trade-Offs

The main trade-off is speed versus coverage. The current suite is fast because it focuses on helper modules. The cost is limited representativeness. Adding controller, workflow, and database integration tests will improve confidence but increase setup complexity and execution time.

The second trade-off is automation versus flexibility. Vite+ provides one command interface and reduces configuration differences, but browser tests, end-to-end tests, and mutation tests still need specialized configuration. A unified toolchain reduces fragmentation. It does not remove all package-specific testing requirements.

The third trade-off is accuracy versus cost. Mutation testing and chaos testing provide stronger evidence than ordinary unit tests, but they are slower and require more controlled environments. For that reason, they are better suited to scheduled or release-level checks than to every commit.

B. Limitations

This study has four limitations. First, it is a single-repository case study, so results cannot be generalized without more projects. Second, the performance dataset is synthetic and small. Third, mutation testing covers only four helper modules. Fourth, controller-level and workflow-level integration tests remain incomplete.

Several unexpected results were useful. The leaderboard route required authentication during real execution even though it was selected as a public aggregate benchmark. The frontend aggregate test command behaved differently from isolated frontend unit execution in the earlier paper inspection. These findings show that composition-level tests are necessary because implementation behavior can differ from local assumptions.

The main lesson is that a QA strategy should not stop at adding tests. Tests must be tied to risks, gates, and measured

outcomes. For Circles, the next improvement should focus on backend integration tests for authenticated routes, stronger validation in the import pipeline, and repeated performance and chaos runs on a larger dataset.

C. Lessons Learned

Three lessons follow from the study. First, risk-based testing is useful only when it changes test selection. The highest-value evidence came from components that were explicitly identified as high risk before testing began. Second, quality gates need scope labels. A coverage value is meaningful only when the covered files are known. Third, experimental tests reveal different information than unit tests. Performance testing exposed route behavior and latency. Mutation testing exposed weak assertions. Chaos testing exposed dependency recovery behavior.

The next iteration should therefore expand breadth before adding more specialized tools. The backend needs integration tests for authenticated controllers, route middleware, and database-backed queries. The frontend needs repeated aggregate test execution to confirm whether the observed composition failure is stable or intermittent. The import pipeline needs stricter tests for Spotify URI types and required metadata fields. After those changes, mutation and performance experiments should be repeated so that the results represent more of the system.

VI. CONCLUSION

This work addressed the problem of fragmented QA evidence in a full-stack TypeScript monorepo. Circles already had automated tests and a unified Vite+ toolchain, but the earlier assignments showed that passing tests alone did not explain whether the most important system risks were covered.

The study proposed a risk-based automated QA strategy. The strategy identified high-impact risks, mapped them to test types, connected those tests to quality gates, and evaluated the system through coverage, execution time, performance, mutation, and chaos experiments. This created a single research narrative from the earlier planning, automation, experimental, and midterm work.

The main findings are realistic rather than absolute. Backend helper tests were fast and stable, with 33 tests passing in the extended suite. The selected mutation scope reached 87.93%, and local authenticated read performance stayed below 25 ms P99. The chaos experiment showed that the backend recovered after PostgreSQL restarted. However, the same experiment also showed a user-visible HTTP 500 during the outage, and global backend coverage stayed below 7%.

The contribution is both methodological and practical. Methodologically, the paper shows how risk-based planning, automation, gates, and experiments can be connected in one QA case study. Practically, it gives the Circles project a clearer test roadmap: strengthen API integration coverage, improve import validation, test authenticated route behavior, and repeat performance and failure-injection experiments on larger data.

The main limitations are the local dataset, narrow mutation scope, incomplete integration coverage, and single-project setting. Future work should add database-backed controller tests, expand Playwright coverage for dashboard and import flows, collect repeated run data for flakiness analysis, and evaluate the same QA framework on a larger deployment-like environment.

Overall, the study shows that the current Circles QA process is useful but not complete. It is useful because the pipeline can already execute tests, collect metrics, and expose real weaknesses. It is incomplete because the strongest evidence is concentrated in helper modules while the most important production risks involve authentication, persistence, and route composition. The final value of the work is therefore not a claim that the system is fully tested. The value is a structured, evidence-based path from risk planning to stronger automated quality assurance.

ACKNOWLEDGMENT

No external funding was used for this coursework paper.

REFERENCES

- [1] B. W. Boehm, "Software Risk Management: Principles and Practices," *IEEE Software*, vol. 8, no. 1, pp. 32–41, 1991, doi: 10.1109/52.62930.
- [2] International Organization for Standardization, "ISO 31000:2018 Risk Management Guidelines." 2018.
- [3] M. Felderer and I. Schieferdecker, "A Taxonomy of Risk-Based Testing," *Software Quality: Model-Based Approaches for Advanced Software and Systems Engineering*. Springer, pp. 121–136, 2014. doi: 10.1007/978-3-319-03602-1_8.
- [4] F. Redmill, "Theory and Practice of Risk-Based Testing," *Software Testing, Verification and Reliability*, vol. 15, no. 1, pp. 3–20, 2005, doi: 10.1002/stvr.319.
- [5] M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, "Usage, Costs, and Benefits of Continuous Integration in Open-Source Projects," in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, ACM, 2016, pp. 426–437. doi: 10.1145/2970276.2970358.
- [6] M. Shahin, M. A. Babar, and L. Zhu, "Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices," *IEEE Access*, vol. 5, pp. 3909–3943, 2017, doi: 10.1109/ACCESS.2017.2685629.
- [7] A. M. Memon *et al.*, "Taming Google-Scale Continuous Testing," in *Proceedings of the 39th International Conference on Software Engineering: Software Engineering in Practice Track*, IEEE, 2017, pp. 233–242. doi: 10.1109/ICSE-SEIP.2017.16.
- [8] Y. Wang, M. V. Mäntylä, Z. Liu, and J. Markkula, "Test Automation Maturity Improves Product Quality: Quantitative Study of Open Source Projects Using Continuous Integration," *Journal of Systems and Software*, vol. 188, p. 111259, 2022, doi: 10.1016/j.jss.2022.111259.
- [9] L. Yu, E. Alégroth, P. Chatzipetrou, and T. Gorschek, "Automated NFR Testing in Continuous Integration Environments: A Multi-Case Study of Nordic Companies," *Empirical Software Engineering*, vol. 28, no. 144, 2023, doi: 10.1007/s10664-023-10356-1.
- [10] D. Stallenberg, M. Olsthoorn, and A. Panichella, "Improving Test Case Generation for REST APIs Through Hierarchical Clustering," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, IEEE, 2021, pp. 117–128. doi: 10.1109/ASE51524.2021.9678586.
- [11] C. Brandt and A. Zaidman, "Developer-Centric Test Amplification: The Interplay Between Automatic Generation and Human Exploration," *Empirical Software Engineering*, vol. 27, no. 4, p. 96, 2022, doi: 10.1007/s10664-021-10094-2.
- [12] L. Inozemtseva and R. Holmes, "Coverage Is Not Strongly Correlated with Test Suite Effectiveness," *Proceedings of the 36th Interna-*

tional Conference on Software Engineering, pp. 435–445, 2014, doi: 10.1145/2568225.2568271.

- [13] Y. Jia and M. Harman, “An Analysis and Survey of the Development of Mutation Testing,” *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011, doi: 10.1109/TSE.2010.62.
- [14] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman, “Mutation Testing Advances: An Analysis and Survey,” *Advances in Computers*, vol. 112, pp. 275–378, 2019, doi: 10.1016/bs.adcom.2018.03.015.
- [15] Z. M. Jiang and A. E. Hassan, “A Survey on Load Testing of Large-Scale Software Systems,” *IEEE Transactions on Software Engineering*, vol. 41, no. 11, pp. 1091–1118, 2015, doi: 10.1109/TSE.2015.2445340.
- [16] C. Barna, M. Litoiu, and H. Ghanbari, “Autonomic Load-Testing Framework,” in *Proceedings of the 8th ACM International Conference on Autonomic Computing*, ACM, 2011, pp. 91–100. doi: 10.1145/1998582.1998600.
- [17] A. Basiri *et al.*, “Chaos Engineering,” in *IEEE International Symposium on Software Reliability Engineering Workshops*, IEEE, 2016, pp. 1–6. doi: 10.1109/ISSREW.2016.21.
- [18] C. Rosenthal, N. Jones, L. Hochstein, and A. Blohowiak, *Chaos Engineering: System Resiliency in Practice*. O’Reilly Media, 2020.
- [19] J. Bogner and M. Merkel, “To Type or Not to Type? A Systematic Comparison of the Software Quality of JavaScript and TypeScript Applications on GitHub,” in *Proceedings of the 19th International Conference on Mining Software Repositories*, ACM, 2022, pp. 658–669. doi: 10.1145/3524842.3528454.
- [20] T. Tang, S. Alimadadi, and N. Sumner, “From Logic to Toolchains: An Empirical Study of Bugs in the TypeScript Ecosystem,” in *Proceedings of the 23rd International Conference on Mining Software Repositories*, ACM, 2026, pp. 1–11. doi: 10.1145/3793302.3793379.